

A.4 - Abstract Classes and Interfaces

To complete our discussion of object-oriented programming, we will discuss ***abstract classes*** and ***interfaces***.

A.4.1 - Abstract Classes

Consider the class hierarchy shown in [Figure A.4.1](#). The Shape class is the parent of the Rectangle and Circle child classes.

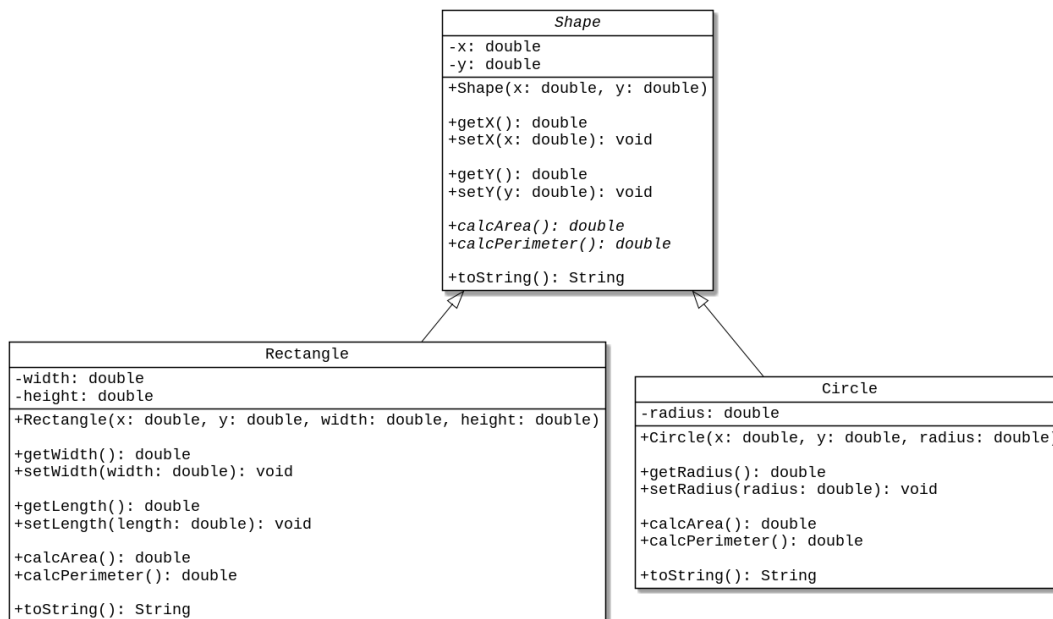


Figure A.4.1. Shape, Rectangle, and Circle classes

A Shape has an x and y location on the screen, you can calculate its area and perimeter, and you can use toString to convert it to a string. Let's start writing the code for this class.

Listing A.4.2.

```
public class Shape {
    private double x;
    private double y;

    public Shape(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return this.x;
    }

    public void setX(double x) {
        this.x = x;
    }

    public double getY() {
        return this.y;
    }

    public void setY(double y) {
        this.y = y;
    }

    public double calcArea() {
        // ?????
    }

    public double calcPerimeter() {
        // ?????
    }

    public String toString() {
        return String.format("(%.1f, %.1f)", this.x, this.y);
    }
}
```

Everything is going great until we reach the `calcArea` and `calcPerimeter` methods. How do we calculate the area and perimeter of a “shape”? We don’t have that sort of problem with circles and rectangles. We know the formulas for calculating their area and perimeter (AKA circumference). But a “shape” really doesn’t exist. If I tell you to draw me a shape, you’ll ask me “which one?” Shapes are an abstraction.

Here's how we'll solve the problem: we will use the keyword `abstract` on the class declaration. This tells Java that we will never directly instantiate a `Shape` object. If anyone tries something like the following, they will get an error message.

```
Shape myShape = new Shape(3.0, 4.5);
```

We will also put the keyword `abstract` on the declarations of `calcArea` and `calcPerimeter`. And, *most important*, we will provide *only* the method header—there will be no method body. [Listing A.4.3](#) shows the changed lines:

Listing A.4.3.

```
public abstract class Shape {  
    private double x;  
    private double y;  
  
    // ...  
  
    public abstract double calcArea();  
  
    public abstract double calcPerimeter();  
  
    public String toString() {  
        return String.format("%.1f, %.1f)", this.x, this.y);  
    }  
}
```

Note A.4.4.

If you look closely at the UML diagram in [Figure A.4.1](#), you will see that the class name and the two method names are in italics. This is how you signify an abstract entity in UML.

Now let's look at the code for the `Circle` class, which extends `Shape`. This is not an abstract class—we know how to calculate a circle's area and perimeter (circumference):

Listing A.4.5. The Circle Class

```
1  public class Circle extends Shape {
2      private double radius;
3
4      public Circle(double x, double y, double radius) {
5          super(x, y);
6          this.radius = radius;
7      }
8
9      public double getRadius() {
10         return this.radius;
11     }
12
13     public void setRadius(double radius) {
14         this.radius = radius;
15     }
16
17     public double calcArea() {
18         return Math.PI * this.radius * this.radius;
19     }
20
21     public double calcPerimeter() {
22         return 2.0 * Math.PI * this.radius;
23     }
24
25     public double calcCircumference() {
26         return calcPerimeter();
27     }
28
29     public String toString() {
30         return String.format("Circle: %s [radius %.1f]", super.toString(),
31             this.radius);
32     }
33 }
```

The class can still use the `super` keyword to call methods in the parent class. Because `Shape` never defined the body of the `calcArea` and `calcPerimeter` methods, the `Circle` class *must* override the methods and provide a method body (lines 17–23). If we omit a definition for `calcArea`, we get this error:

```
Circle.java:1: error: Circle is not abstract and does not override
abstract method calcArea() in Shape
public class Circle extends Shape {
    ^
1 error
```

Note A.4.6.

Even though you cannot directly instantiate a `Shape`, you can use polymorphism to create, for example, an array of `Shape` and assign concrete (non-abstract) objects such as `Circle` and `Rectangle` to its elements, as in [Listing A.4.7](#).

Listing A.4.7.

```
public class ShapeExample {
    public static void main(String[] args) {
        Shape[] shapeList = {
            new Circle(3.0, 4.0, 1.5),
            new Rectangle(1.5, 2.5, 4.7, 8.3)
        };
        for (Shape s: shapeList) {
            System.out.printf("%s area: %.2f perimeter: %.2f\n",
                s, s.calcArea(), s.calcPerimeter());
        }
    }
}
```

A.4.2 - Interfaces

Let's look at the simplified UML diagram showing the inheritance for the `Bicycle`, `ElectricBicycle`, and `CargoBicycle` classes from [Exercise A.3.1](#).

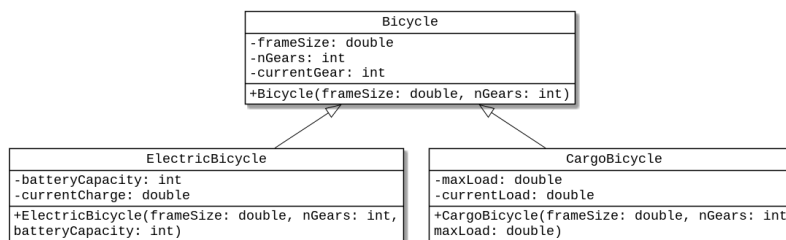


Figure A.4.8. Inheritance of `Bicycle`, `ElectricBicycle`, and `CargoBicycle`

What if we want to have an electric cargo bicycle? We would like to be able to say something like the following:

```
public class ElectricCargoBicycle extends ElectricBicycle, CargoBicycle {
    // ...
}
```

This is called **multiple inheritance**, and its UML diagram would look something like this:

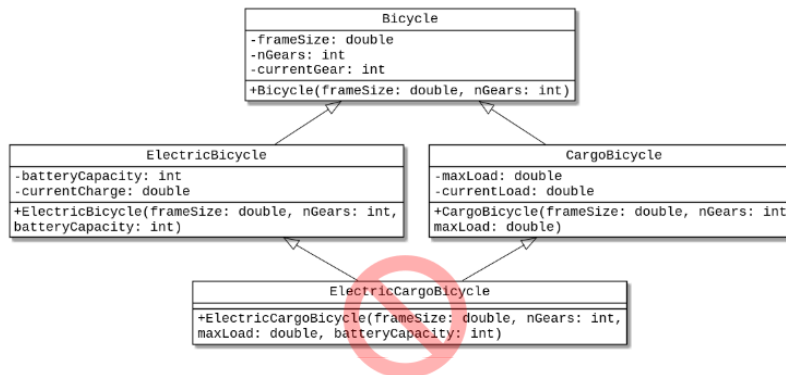


Figure A.4.9. Inheritance of ElectricCargoBicycle

But we can't! Java does not allow multiple inheritance.

One solution to this problem is to use an **interface**. It is somewhat like an abstract class, except that it consists of only methods. [Listing A.4.10](#) shows the Electrified interface:

Listing A.4.10.

```
interface Electrified {
    public int getChargeCapacity();
    public double getCurrentCharge();
    public void setCurrentCharge(double charge);
}
```

Let's look at the Bicycle class in [Listing A.4.11](#):

Listing A.4.11.

```
class Bicycle {
    private double frameSize;
    private int nGears;
    private int currentGear;

    Bicycle(double frameSize, int nGears) {
        this.frameSize = frameSize;
        this.nGears = nGears;
        setCurrentGear(nGears);
    }

    public double getFrameSize() {
        return this.frameSize;
    }

    public int getNGears() {
        return nGears;
    }

    public int getCurrentGear() {
        return currentGear;
    }

    public void setCurrentGear(int currentGear) {
        this.currentGear = Math.min(this.nGears, Math.max(currentGear, 1));
    }
}
```

Now, our ElectricBicycle class lookss like this:

Listing A.4.12.

```
1  class ElectricBicycle extends Bicycle implements Electrified {
2      private int chargeCapacity;
3      private double currentCharge;
4
5      ElectricBicycle(double frameSize, int nGears, int chargeCapacity) {
6          super(frameSize, nGears);
7          this.chargeCapacity = chargeCapacity;
8      }
9
10     /* Implement the methods in the interface */
11     public int getChargeCapacity() {
12         return this.chargeCapacity;
13     }
14
15     public double getCurrentCharge() {
16         return this.currentCharge;
17     }
18
19     public void setCurrentCharge(double charge) {
20         this.currentCharge = Math.max(0, Math.min(charge, chargeCapacity));
21     }
22 }
```

Line 1 is the key here: the class `implements` the interface. Because this class implements `Electrified`, it must provide method bodies for all the methods defined in the interface; those are in lines 11–21.

We can now create our `ElectricCargoBicycle` by using an interface. We extend `CargoBicycle` and implement `Electrified`:

Listing A.4.13.

```

class ElectricCargoBicycle extends CargoBicycle implements Electrified {
    private int chargeCapacity;
    private double currentCharge;

    public ElectricCargoBicycle(double frameSize, int nGears,
        double maxLoad, int chargeCapacity) {
        super(frameSize, nGears, maxLoad);
        this.chargeCapacity = chargeCapacity;
        this.currentCharge = 0.0;
    }

    /* Implement the methods in the interface */
    public int getChargeCapacity() {
        return this.chargeCapacity;
    }

    public double getCurrentCharge() {
        return this.currentCharge;
    }

    public void setCurrentCharge(double charge) {
        this.currentCharge = Math.max(0, Math.min(charge, chargeCapacity));
    }
}

```

Figure A.4.14 shows the relationship among these classes.

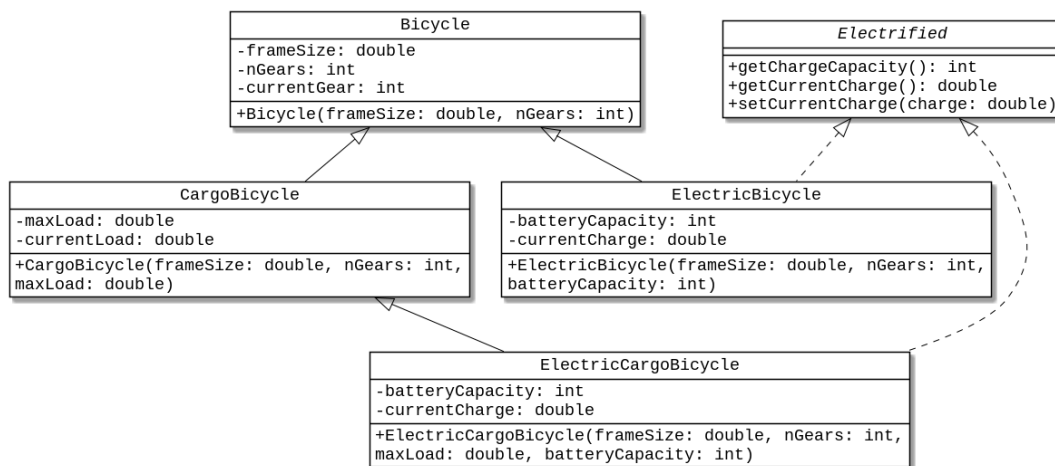


Figure A.4.14. Bicycle and its Subclasses

Note A.4.15.

In this particular example, we could also use composition to create a Battery class and put one of those objects into the electric bicycle and electric cargo bicycle classes.

Java makes extensive use of interfaces in much of its class library. For example, the `ArrayList` class implements these interfaces:

- `Iterable`, which provides methods for enabling iteration via enhanced for loops
- `Collection`, which provides methods for adding and removing elements
- `List`, which provides methods for integer indexing

You have attempted 1 of 1 activities on this page.

Bruno
The Git-native
API client



Git-native API client
Send requests to REST,
GraphQL and gRPC.
Collections live in git.
Get Bruno
usebruno.com

Ads by EthicalAds